Dos métodos de clases relacionadas al saldo del monedero.

userSessionGetSaldoDespuesDeIniciar ()

Tabla especificación:

Entradas	Precondiciones	Especificaciones	Salidas
Objeto Usuario con saldo inicial	El sistema no tiene una sesión activa previa o permite sobrescribirla.	Sesión.getSaldo(): Debe retornar el valor El valor numérico exacto del saldo del usuario que inicio sesión.	Float (Saldo igual al del usuario)

Tabla clases de equivalencia

Factor	Clases de Equivalencia	Entrada	Salida
Saldo Positivo	[0.01,max)	100.0f	100.0f
Saldo Cero	{0,0}	0.0f	0.0f
Saldo Negativo	(-min,-0.01]	-50.0f	-50.0f

userSessionReflejaCambiosEnElSaldoDelUsuario ()

Tabla especificación:

Entradas	Precondiciones	Especificaciones	Salidas
Objeto Usuario y métodos AddSaldo / SubSaldo	Sesión iniciada con dicho objeto Usuario.	session.getSaldo(): Debe retornar el saldo actualizado del usuario tras operaciones de suma o resta.	float (Saldo final calculado)

Tabla clases de equivalencia

Factor	Clases de Equivalencia	Entrada	Salida
Incremento	Suma de valores positivos	100.0f + addSaldo(10.0f)	110.0f
Decremento	Resta de valores positivos	110.0f + subSaldo(50.0f)	60.0f
Límite (Cero)	Resta del total del saldo	60.0f + subSaldo(60.0f)	0.0f
Saldo Negativo	Resta mayor al saldo actual	0.0f + subSaldo(10.0f)	-10.0f

Dos métodos de clases relacionadas al cálculo del CCB

ccbCalculaPrecioBaseCorrectamente ()

Tabla especificación:

Entradas	Precondiciones	Especificaciones	Salidas
Cuatro valores numéricos (doublé): v1, v2, v3, v4	El tercer parámetro(v3) debe ser distinto de cero para evitar error de división.	Calcula el precio base sumando los dos primeros valores, dividendo por el tercero y aplicando el porcentaje del cuarto.	Doublé(resultado del cálculo con precisión de 0.01)

Tabla clases de equivalencia:

Factor	Clases de Equivalencia	Entrada	Salida
Calculo Estandar	Valores positivos tipicos	100.0 , 50.0,10.0,20.0	18.0
Sin Incremento	Porcentaje (v4) igual a 0	100.0,50.0,10.0,0.0	15.0
Numerador Cero	Suma de v1 + v2 = 0	0.0,0.0,10.0,20.0	0.0
Valores Decimales	Entradas con precisión	10.5,4.5,3.0,10.0	5.5

ccbCalculadoRedondeaA2Decimales ()

Tabla especificación:

Entradas	Precondiciones	Especificaciones	Salidas
Parámetros Doublé: v1,v2,v3,v4	El divisor (v3) no es cero.	Aplica la fórmula de CCB y redondea el resultado al segundo decimal más cercano(redondeo simétrico/estándar).	Doublé con exactamente 2 decimales.

Tabla clases de equivalencia:

Factor	Clases de Equivalencia	Entrada	Salida
Redondeo hacia arriba	3er decimal >=5	55.22,10.33,7.0,13.5	10.63
Redondeo hacia abajo	3er decimal <5	10.624	10.62
Sin necesidad de redondeo	Resultado con <= 2 decimales	10.5	10.50
Valores Entero	Resultado sin decimales	Un cálculo que resulte en, por ejemplo 18.0	18.00

***Un método de clases relacionadas a la configuración de Menú**

actualizarMenuItemConExitoDevuelveTrue()

Tabla especificación:

Entradas	Precondiciones	Especificación	Salidas
id, nombre, desc, img, horario	El ítem con el id especificado debe existir previamente en el modelo.	actualizarMenuItem(...): si id existe $\rightarrow$ actualiza datos y retorna true si id no existe $\rightarrow$ retorna false	true y el objeto MenuItem con los valores modificados.

Tabla clases de equivalencia

Factor	Clases de Equivalencia	Entrada	Salida
Existencia de Registro	ID coincidente con un ítem existente	id del item "Pasta"	true
Integridad de Datos	Nuevos valores válidos para los campos	"Cena de Pasta", "Nueva descripcion", etc.	assertEquals -> True
Estado del Modelo	El objeto original es reemplazado o modificado	model.buscarPorId(id)	updatedItem con datos nuevos

registrarUsuarioYAutenticar()

Tabla especificación:

Entradas	Precondiciones	Especificación	Salidas
id, password	El usuario debe estar registrado en el sistema (UCVSecDB.csv).	**autenticar(id, pass):** si credenciales coinciden valida el Usuario si credenciales NO coinciden no valida y arroja null	Objeto Usuario o null.

Tabla clases de equivalencia

Factor	Clases de Equivalencia	Entrada	Salida
Credenciales	Credenciales correctas (ID y Password coinciden)	id: "123", pass: "secret"	Objeto Usuario (Not Null)
Password	Credenciales incorrectas (Password erróneo)	id: "123", pass: "bad"	null
Rol de Usuario	Usuario con rol asignado en DB	autenticado.getRole()	"Administrador"

autenticarUsuarioNoExistenteDevuelveNull()

Tabla especificación:

Entradas	Precondiciones	Especificación	Salidas
id, password	El UserModel se inicializa sin registros de usuarios previos.	autenticar(id, pass): si id no existe en DB entonces retorna null si id existe entonces retorna Usuario	null

Tabla clases de equivalencia

Factor	Clases de Equivalencia	Entrada	Salida
Existencia en DB	ID no registrado en el sistema	`id: "doesnotexist"`	`null`
Estado de Credencial	Cualquier contraseña para ID inexistente	`pass: "pwd"`	`null`